Data, Computation & Innovation II

Instructor: Jared Whalen

Housekeeping

- Coursework due dates
- Video walkthroughs

Course Overview

Week 1 (June 3): Chart Fundamentals + SVG/Svelte Introduction

Week 2 (June 10): D3 Fundamentals, Part 1

Week 3 (June 17): D3 Fundamentals, Part 2

Week 4 (June 24): Interactivity and animation ***+ Guest speaker Kavya Beheraj/The Athletic***

Week 5 (July 1): GIS and Mapping *+ Guest speaker Alvin Chang/The Pudding*

Week 6 (July 8): Scrollytelling + Advanced patterns

Week 7 (July 15): Final project presentations

✨ Review ✨

- Name three D3 shape generators.
- In your own words, what is the difference between the **declarative** and **imperative** methods of building charts with D3 + Svelte?
- What does the JavaScript `.map(d => …)` function do to an array?
- In Svelte, what is the difference between the `$state()` and `$derived()` rune?

Critique (~10 minutes)

Break into groups of ~3 and discuss last week's assignment

- **Author:** Explain what you did and why
- **Editor:** Give specific questions, give specific feedback
  - what's working?
  - what's unclear?
- **Everyone:** What edits would resolve these tensions?

Interactivity and animation

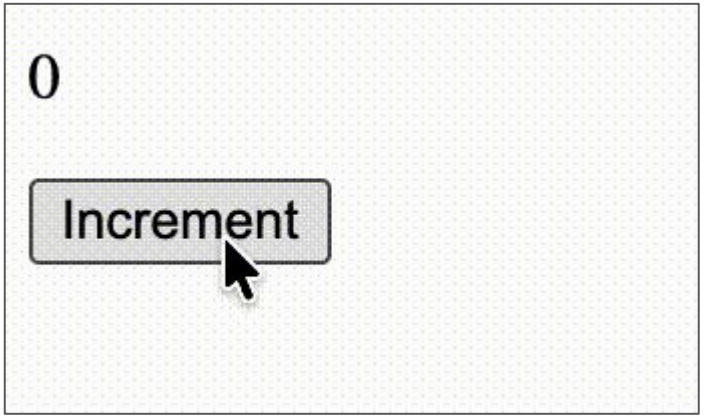
State in Svelte

Everything you've built so far has been static — data declared once, chart rendered once. Interactivity means your component can hold values that change over time, and the chart updates when they do.

In Svelte, reactive state is declared with `$state()`.

`$state()` creates a **rune** — a reactive value. When it changes, anything that depends on it re-renders automatically. You don't call a setter, you don't trigger an update — you just reassign.

```
<script>
  let count = $state(0)
</script>

<p>{count}</p>
<button onclick={() => count++}>Increment</button>
```

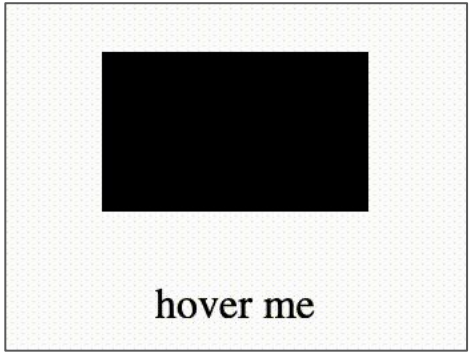


Events

Svelte uses standard DOM event attributes. The `onclick`, `onmouseover`, `onmouseout`, `oninput` — they work the same way on HTML elements and SVG elements.

The **handler** is just a function. Reassign state inside it and the component updates.

```
<script>
    let message = $state('hover me');
</script>

<svg>
    <rect
        x="50"
        y="50"
        width="100"
        height="60"
        onmouseenter={() => (message = 'inside')}
        onmouseleave={() => (message = 'outside')}
    />
    <text x="100" y="150" text-anchor="middle">{message}</text>
</svg>
```



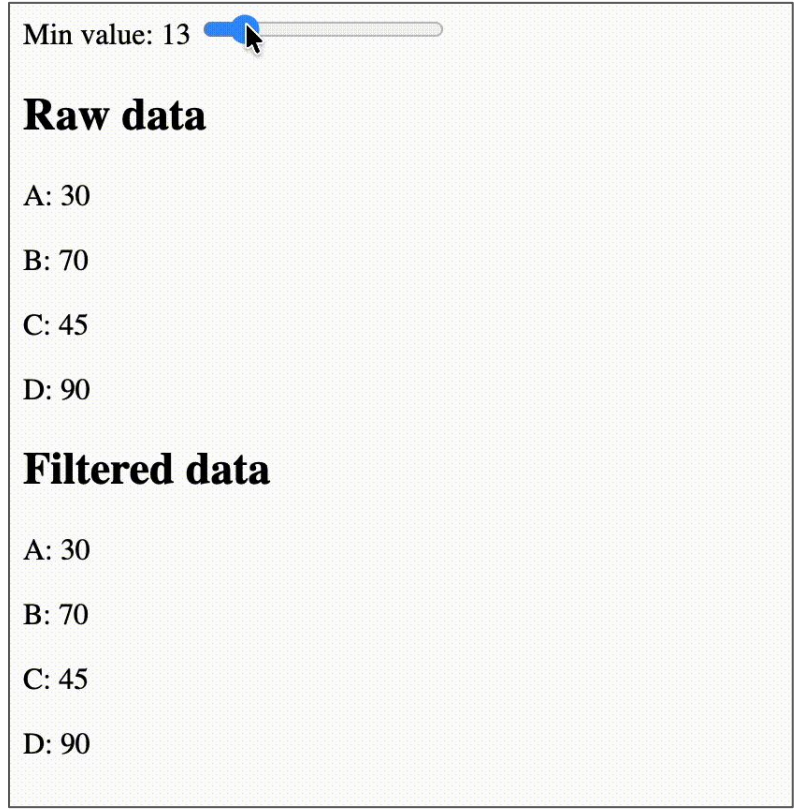
Derived state with `$derived`

When a value is computed from other state, use `$derived` instead of recalculating in the template.

When `threshold` changes, `filtered` recalculates. The filtered list updates immediately — no event handler, no manual re-render. This is the core pattern for interactive filtering, and the same chain extends to scales and chart elements in the next example.

```
<script>
  let threshold = $state(50)

  const data = [
    { label: "A", value: 30 },
    { label: "B", value: 70 },
    { label: "C", value: 45 },
    { label: "D", value: 90 }
  ]

  let filtered = $derived(data.filter(d => d.value >= threshold))
</script>
```

```svelte
<script>
  let threshold = $state(50)

  const data = [
    { label: "A", value: 30 },
    { label: "B", value: 70 },
    { label: "C", value: 45 },
    { label: "D", value: 90 }
  ]

  let filtered = $derived(data.filter(d => d.value >= threshold))
</script>

<label>
  Min value: {threshold}
  <input type="range" min="0" max="100" bind:value={threshold} />
</label>

<h2>Raw data</h2>
{#each data as d}
  <p>{d.label}: {d.value}</p>
{/each}

<h2>Filtered data</h2>
{#each filtered as d}
  <p>{d.label}: {d.value}</p>
{/each}
```

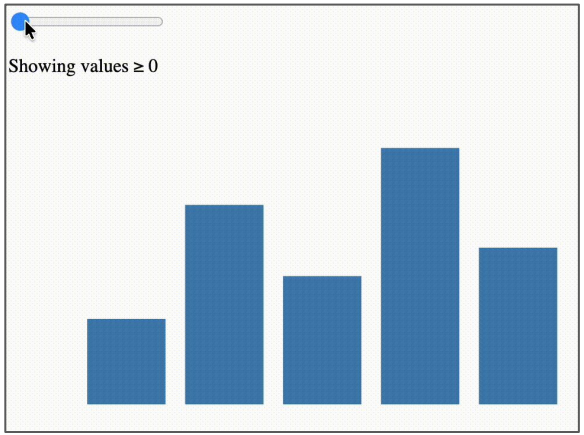


Connecting state to a chart

When using Svelte's reactivity and D3 together, your scales, generators, and chart attributes all recalculate when state changes. Nothing special is required — they're just expressions that depend on reactive values.

The slider updates `threshold`, which updates `data`, which updates the scales, which updates the rects. The whole chain is reactive.

```
let data = $derived(allData.filter(d => d.value >= threshold))

let xScale = $derived(
  d3.scaleBand()
    .domain(data.map(d => d.label))
    .range([margin.left, width - margin.right])
    .padding(0.2)
)

let yScale = $derived(
  d3.scaleLinear()
    .domain([0, 100])
    .range([height - margin.bottom, margin.top])
)
```



A quick note on bindings: `bind:value`

`bind:value` is Svelte's two-way binding shorthand for elements with a `value` attribute. Instead of writing an `oninput` handler that updates state, you wire the input's value directly to a reactive variable.

Both work. `bind:value` is less code. Use it for form controls where the value is the only thing you care about.

```svelte
<!-- with bind -->
<input type="range" bind:value={threshold} />

<!-- equivalent without bind -->
<input
  type="range"
  value={threshold}
  oninput={(e) => threshold = Number(e.target.value)}
/>
```

A quick note on bindings: `bind:clientWidth`

We can also bind the element values, such as dimensions. Svelte's `bind:clientWidth` lets you read the rendered width of any element and use it as reactive state.

The div wraps the SVG. `bind:clientWidth` writes the div's rendered pixel width into `containerWidth` as soon as it mounts, and again whenever the browser resizes. Everything that depends on `containerWidth` — scales, generators, axis ticks — recalculates automatically.

One note: `containerWidth` starts at 0 before the component mounts, so the first render produces a zero-width chart. Guard against it with an {#if}

```
    let containerWidth = $state(0);
</script>

{#if containerWidth > 0}
    <div bind:clientWidth={containerWidth}>
        <svg width={containerWidth} {height}>
            <!-- chart -->
        </svg>
    </div>
{/if}
```

Updating attributes reactively

The same pattern applies to any SVG attribute — including fill. Connect color to state and it updates like anything else.

```
const colorScale = d3.scaleOrdinal()
  .domain(data.map(d => d.label))
  .range(d3.schemeTableau10)

let selected = $state(null)
```

```
<!-- Legend -->
<div class="legend">
  {#each data as d}
    <button
      onclick={() => selected = selected === d.label ? null : d.label}
      style="border-color: {colorScale(d.label)}"
      class:active={selected === d.label}
    >
      {d.label}
    </button>
  {/each}
</div>
```

```
<!-- Columns -->
{#each data as d}
  <rect
    x={xScale(d.label)}
    y={yScale(d.value)}
    width={xScale.bandwidth()}
    height={height - margin.bottom - yScale(d.value)}
    fill={colorScale(d.label)}
    opacity={selected === null || selected === d.label ? 1 : 0.2}
  />
{/each}
```

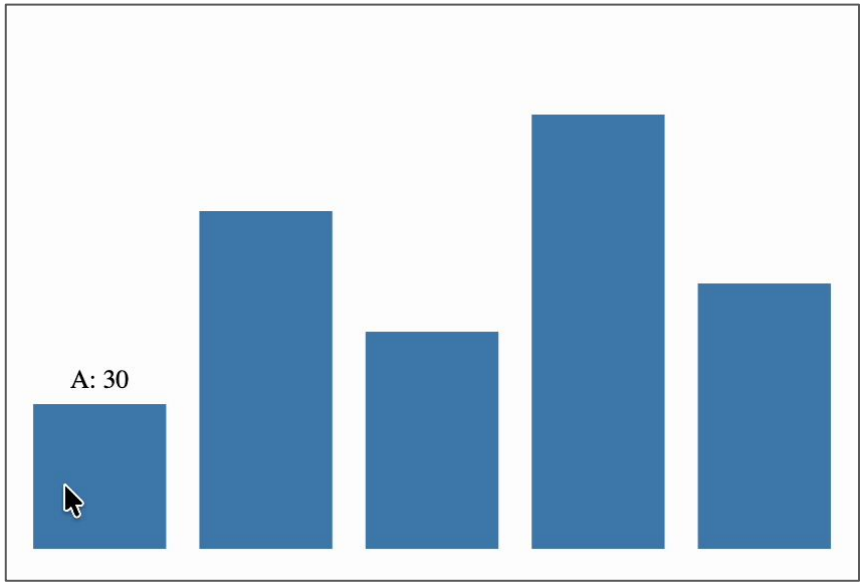
```
const colorScale = d3.scaleOrdinal()
  .domain(data.map(d => d.label))
  .range(d3.schemeTableau10)

let selected = $state(null)
```

```
<!-- Legend -->
<div class="legend">
  {#each data as d}
    <button
      onclick={() => selected = selected === d.label ? null : d.label}
      style="border-color: {colorScale(d.label)}"
      class:active={selected === d.label}
    >
      {d.label}
    </button>
  {/each}
</div>
```

```
<!-- Columns -->
{#each data as d}
  <rect
    x={xScale(d.label)}
    y={yScale(d.value)}
    width={xScale.bandwidth()}
    height={height - margin.bottom - yScale(d.value)}
    fill={colorScale(d.label)}
    opacity={selected === null || selected === d.label ? 1 : 0.2}
  />
{/each}
```

Tooltips

A tooltip is just a conditional element positioned with reactive state. We can use `onmouseenter` as the handler and the existing scales to position the new `text` element.




```
let tooltip = $state(null);
// tooltip will hold { x, y, label, value } or null
```

```
<svg {width} {height}>
    {#each data as d}
        <rect
            x={xScale(d.label)}
            y={yScale(d.value)}
            width={xScale.bandwidth()}
            height={height - margin.bottom - yScale(d.value)}
            fill="steelblue"
            onmouseenter={(e) => {
                tooltip = {
                    x: xScale(d.label) + xScale.bandwidth() / 2,
                    y: yScale(d.value) - 8,
                    label: d.label,
                    value: d.value
                };
            }}
            onmouseleave={() => (tooltip = null)}
        />
    {/each}

    {#if tooltip}
        <text
            x={tooltip.x} y={tooltip.y}
            text-anchor="middle"
            dominant-baseline="auto"
            font-size="13"
        >
            {tooltip.label}: {tooltip.value}
        </text>
    {/if}
</svg>
```

Other tooltip approaches

Following the cursor

Instead of anchoring the tooltip to the data point's scale position, you can track the mouse position directly using onmousemove on the SVG. `offsetX` and `offsetY` give the cursor position relative to the element.

```
<svg
    onmousemove={(e) => {
        tooltipX = e.offsetX;
        tooltipY = e.offsetY
    }}
>
</svg>
```

HTML overlays

For richer tooltips (formatted numbers, multiple lines, icons) an absolutely positioned HTML `<div>` gives you full CSS control. The SVG and the div sit in the same container, the div is positioned with position: absolute, and you update its left and top from the same mouse event data.

Tippy.js

Tippy.js is a lightweight tooltip library that handles positioning, edge detection, and animation out of the box. You attach it to any DOM element and pass it content. It's a reasonable choice when you want polished tooltips without building the positioning logic yourself.

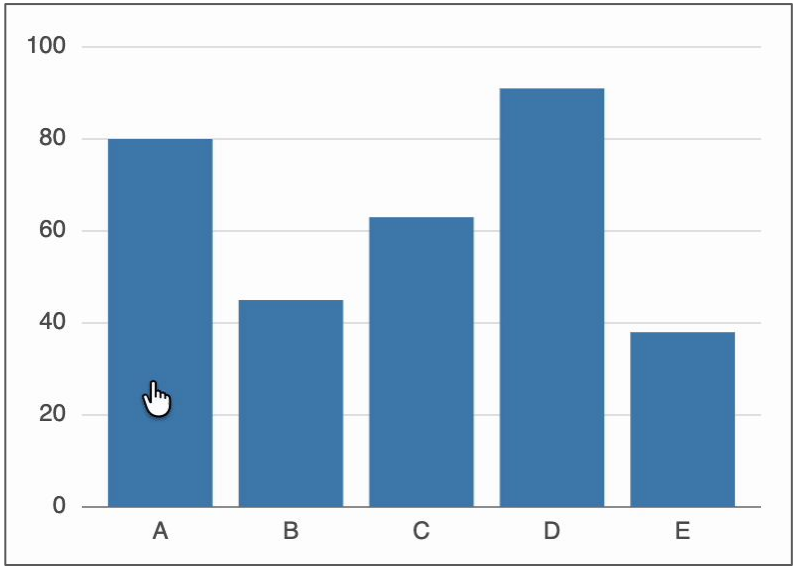
CSS transitions on SVG

SVG elements are part of the DOM, which means CSS transitions work on them.

The `transition` CSS property is a shorthand property for the various transition settings such as `transition-property` and `transition-duration` (MDN docs).

Any time fill or opacity changes, the browser animates between the old and new values. No JavaScript animation logic. No d3.transition(). Just CSS.

```
<style>
    …
    rect {
        transition:
            fill 200ms ease,
            opacity 200ms ease;
    }
</style>
```



What CSS transitions can and can't animate

Works:

- `fill`
- `stroke`
- `opacity`
- `transform` (translate, scale, rotate)
- `r` (circle radius — in most browsers)

Doesn’t work (directly):

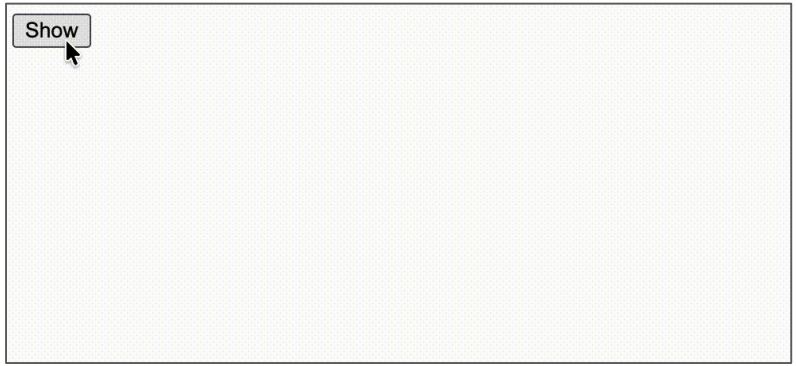
- `x`, `y`, `width`, `height`, `d` as HTML attributes (use `transform` instead, or set them as CSS properties)

Svelte transitions

Svelte has built-in transitions for elements **entering** and **leaving** the DOM: `fade`, `fly`, `slide`, `scale`, and more. They trigger automatically on `{#if}` toggles and keyed `{#each}` blocks.

These work well for tooltips, annotations, and labels — elements that appear and disappear based on interaction.

```svelte
<script>
    import { fade, fly } from 'svelte/transition';

    let show = $state(false);
</script>

<button onclick={() => (show = !show)}>
    {show ? 'Hide' : 'Show'}
</button>

{#if show}
    <div transition:fade={{ duration: 300 }}>
        <p>This fades in and out.</p>
    </div>

    <div in:fly={{ y: 20, duration: 400 }} out:fade={{ duration: 200 }}>
        <p>This flies in from below, fades out.</p>
    </div>
{/if}
```



A quick note on `each` block transitions and `{#key}`

When items in an `{#each}` block change, Svelte updates the existing elements in place — it doesn't recreate them, so `transition:` won't fire on data changes.

If you want elements to animate when the underlying data changes (filtering, sorting), use `{#key}` to force a re-mount.

In this example, the `(d.label)` is a keyed `{#each}`. Svelte tracks each element by its key. When an item leaves the array, Svelte runs its `out:` transition before removing it. When a new item appears, it runs the `in:` transition.

```
{#each data as d (d.label)}
  <g transition:fade={{ duration: 150 }}>
    <rect ... />
  </g>
{/each}
```

Putting it together

- **`$state`** — a reactive variable. Holds whatever changes over time: which item is selected, what the tooltip should show, the current filter value.
- **`$derived`** — a value computed from state. Use it for filtered datasets and scales that depend on state. Recalculates automatically when its dependencies change.
- **Event handlers** — standard DOM events on SVG elements. onclick for selection, `onmouseenter` and `onmouseleave` for hover behavior.
- **`bind:value`** — two-way binding for form controls. Wires a range slider or dropdown directly to a state variable without writing an event handler.
- **CSS transitions** — declared in `<style>`, animate property changes on persistent elements. Use for fill, opacity, and stroke changes that happen in place.
- **Svelte transitions** — `fade`, `fly`, and others from `svelte/transition`. Animate elements entering and leaving the DOM. Use for tooltips appearing, filtered items exiting.

 Build Period